GUI Agent 调研报告

1 报告概述

本报告重点梳理了近两年（2025-2026）GUI Agent 领域的最新研究进展，包括基本定义、平台差异、评估体系、训练方法、架构设计、产业实践、问题挑战。旨在为进一步研究提供参考。

2 定义

2.1 什么是 GUI Agent

GUI Agent 是一种自主或半自主系统，通过观察人类面向的图形界面，选择并执行动作以完成用户任务目标。与调用 API 的传统自动化不同，GUI Agent 直接操作可视化界面，模拟人类的点击、输入、滑动、拖拽等操作。

问题建模：

在部分可观测马尔可夫决策过程（POMDP）框架下，一个问题可以定义为一个六元组：

$\mathcal{E}_p = (\mathcal{S}_p, \mathcal{O}_p, \mathcal{A}_p, T_p, \Omega_p, C_p)$

其中 $p \in \{\text{web}, \text{mobile}, \text{desktop/OS}\}$ 表示平台类型，各参数含义如下：

符号	名称	含义	举例
$\mathcal{S}_p$	状态空间	系统所有可能的底层状态，包括界面可见部分和隐藏部分	当前页面、表单内容、登录状态、弹窗是否出现，以及未被 DOM 暴露的后台状态
$\mathcal{O}_p$	观察空间	Agent 能够实际获取的信息	屏幕截图、DOM 树、URL、HTML 源码
$\mathcal{A}_p$	动作空间	Agent 能够执行的操作	点击按钮、输入文本、滑动页面、拖拽元素、按键、返回
T_p	状态转移函数	执行动作后系统状态如何变化	点击"提交"后页面跳转，输入文本后表单更新，滑动后列表滚动
Ω_p	观察函数	从底层状态到可见观察的映射	底层页面状态经渲染后，Agent 只看到一张截图或一棵 DOM 树
C_p	成本或风险函数	每个动作的执行代价或副作用风险	误点"删除"造成数据丢失，重复点击浪费步数，输入错误信息导致任务失败

Agent 维护一个**内部记忆** b_t，依据策略 π_θ 选择动作：

$$a_t \sim \pi_\theta(a \mid x, o_{<t}, a_{<t}, b_t)$$

其中 x 是用户指令，$o_{<t}$ 是之前所有观察，$a_{<t}$ 是之前所有动作，b_t 是 Agent 对历史信息和当前任务状态的内部记忆。

2.2 能力架构

六层能力架构：

感知（Perception）→ 定位（Grounding）→ 规划（Planning）→ 记忆（Memory）→ 执行（Execution）→ 验证（Verification）

层次	核心问题	代表方法
感知	观察的内容	截图、DOM、可访问性树、混合解析
定位	指令到界面目标元素的映射	SeeClick、CogAgent、UGround、ShowUI
规划	子目标分解与长程推理	ReAct、层次规划、树搜索
记忆	历史状态保持与压缩	摘要压缩、KV 缓存、连续记忆
执行	动作空间与执行方式	点击/输入、代码生成、工具调用
验证	执行结果检查与恢复	WebArena Verified、VeriSafe、WorkBench

3 平台差异

Web、移动端、桌面/OS 三类平台在隐藏状态、动作语义、执行成本和不确定性来源上存在显著差异，因此评估体系、架构设计与恢复策略均需按平台分别设计，不能用统一标准衡量所有平台。

3.1 平台差异总览

维度	Web	移动端	桌面/OS
隐藏状态	后端事务状态：认证、配额、库存、服务端验证、事务提交	设备上下文与权限：权限、账户、通知、缓存偏好、App 内存	文件系统与窗口焦点：文件路径、窗口焦点、文件版本、用户账户、附件元数据
动作语义	结构化元素操作：点击 DOM、输入文本、选择选项，语义较明确	手势语义压缩：下滑可能是滚动/打开通知栏/刷新/关闭弹窗	跨应用工作流：一个应用复制、另一个粘贴，语义是应用间状态流转

执行成本	中等：页面加载、网络请求；多数可逆，支付/提交不可逆	高：隐私、延迟、中断、资源受限	高：不可逆操作、跨应用副作用、回滚困难、高分辨率定位
主要不确定性	可见页面与后端状态之间的差距	设备状态的个性化与动态变化	跨应用协调与不可逆操作
应对策略	验证后端是否真的发生预期改变	适应高度个性化、动态变化的设备环境	处理结构异质性，具备回滚和恢复能力
典型失败	视觉成功但事务失败	相同手势触发不同行为	局部正确但全局错误
失败本质	把视觉反馈当成事务完成的证据	未考虑设备上下文对动作语义的影响	未维护全局工作流状态一致性

3.2 代表基准

不同平台在隐藏状态、动作语义与执行成本上差异显著，因此也发展出各自的测试基准，分别评估 Web、移动端和桌面/OS 环境下的 GUI Agent 能力。

平台	基准	说明
Web	WebArena	自托管高仿真 Web 环境，复刻电商、论坛、GitLab 等，自动检查后端状态
Web	Mind2Web	137 个真实网站的 2000+ 条人类操作轨迹，测试未见网站上的泛化能力
移动端	AndroidWorld	功能完整的 Android 模拟环境，20 个 App 中的 116 个可编程任务
移动端	AitW	71.5 万条真实 Android 操作轨迹，训练和评估手势控制能力
桌面/OS	OSWorld	真实虚拟机中的系统级评测平台，数百个跨应用开放式任务
桌面/OS	WorkArena / Windows Agent Arena	企业知识工作基准 / Windows 专用基准

4 评估与基准

4.1 基准分类

现有基准大致可分为三类：静态数据集、交互式环境和结果导向评估。

静态数据集

一个固定不变的样本集合。每条样本包含任务指令、界面截图、人类标注的正确动作序列。Agent 在不与环境交互的情况下预测下一步动作，与标准答案比对。评估的是**单步预测能力**。

基准	内容	评估方式
Mind2Web	137 个真实网站的 2000+ 任务轨迹	比对元素选择、动作类型、输入值是否正确
ScreenSpot	指令 + 截图，要求输出目标元素坐标	预测坐标是否落在标注边界框内
AitW	71.5 万条真实 Android 操作轨迹	训练和评估手势控制能力，纯视觉依赖

优势：轻量、易复现、成本低。

局限：无法反映动态环境中的状态变化。

交互式环境

可执行、有状态的动态系统。Agent 执行动作后环境状态真实改变，观察随之更新，任务是否完成由环境本身判定。问题被建模为**部分可观察的多步决策过程**。

基准	内容	评估方式
WebArena	自托管高仿真 Web 环境，复刻电商、论坛、GitLab	直接检查后端状态是否发生预期改变，而非仅看页面反馈
AndroidWorld	20 个真实 App 中的 116 个任务	通过程序化任务构建与自动检查，判断任务目标是否达成
OSWorld	真实虚拟机中的数百个跨应用开放式任务	基于执行结果评估，直接检查文件、设置等副作用是否满足预期

优势：交互式环境会让 Agent 在接近真实的系统中执行动作，因此能暴露出静态数据集中看不到的问题，例如状态会真实变化、任务可能被中断、还存在一些隐含前提。这使得评估更接近 Agent 实际部署时的情况。

局限：环境搭建与配置复杂，需要大量存储和工程能力。

结果导向评估

结果导向评估是交互式环境的进一步演进。它不仅严格检查底层状态是否真的发生预期改变，还关注完成任务所需的步骤数与时间，以及执行过程是否可验证、可追溯。

基准	内容	核心贡献

WebArena Verified	对 WebArena 的评估器做审计和增强	证明可靠评估必须做事后检查，不能只看页面反馈；评估器设计本身也是研究问题
WorkBench	面向真实职场工作流的结果导向评估	强调局部动作正确，不等于整个任务成功
OSWorld-Human	对比 Agent 和人类的成功率与效率	发现 Agent 需要的步骤数是人类的 2.7–4.3 倍，反思时间占延迟的 76%–96%

4.2 评估指标的层次

层次	指标	定义	判定方式
结果	任务成功率	最终状态满足目标即成功	状态检查 / LLM-judge / 人工
	进度得分	完成子目标数 / 总子目标数	子目标检查
过程	步骤得分	正确步骤数 / 总步骤数	参考轨迹
	UI 定位准确率	预测坐标与真实元素 / 命中率	标注框
	恢复成功率	失败后成功恢复并完成比例	轨迹分析
效率	交互效率	完成任务所需步数	轨迹计数
	延迟 / Token / 成本	端到端时间、token、GPU 开销	系统日志
	人工干预频率	人工确认/接管/修复次数	交互日志
安全	攻击成功率	恶意内容诱导错误动作比例	对抗测试
	隐私泄露率	敏感信息被读取/外传比例	审计日志
	策略合规率	遵守安全/业务策略比例	策略检查
部署	人类一致性	自动评估与人类判断一致比例	人工标注
	可用性 / 可审计性	用户评分、日志可审计程度	用户研究 / 审计

5 训练方法

5.1 强化学习的三种范式

文献将 GUI Agent 的 RL 方法分为三大范式，核心区别在于是否与真实环境交互：

范式	数据来源	代表算法	核心优势	核心问题
离线 RL	固定数据集	DPO、IQL、CQL	安全、可扩展、无环境成本	分布偏移、价值高估
在线 RL	实时环境交互	PPO、GRPO	自适应、可发现更优策略	样本效率低、I/O 延迟高、有风险
混合策略	离线预热 + 在线微调	GRPO + 行为克隆	兼顾安全与适应	工程复杂度高

5.2 奖励工程

理想的奖励由最终结果分与过程引导分两部分构成，但过程分的设计是核心难题：

$$\mathcal{R}^*(s_t, a_t) = \underbrace{\mathcal{V}[\text{task completed}]}_{\text{最终结果分}} + \underbrace{\lambda \cdot \phi(s_t, a_t, g)}_{\text{过程引导分}}$$

层级	类型	代表	优势	局限
底层	规则奖励	UI-R1、BTL-UI、GUI-G^2	不可作弊的客观真值	稀疏、一像素偏差等同全错
中层	学习奖励	GUI-G^2 高斯、InfiGUI-G1 AEPO	密集信号、几何归纳偏置	可能过拟合表面视觉线索
顶层	LLM-as-Judge	ProRe、SmartSnap、ZeroGUI、WebRL	处理模糊语义任务	成本高、幻觉、奖励黑客

6 架构设计

6.1 框架论文与模块统计

框架论文指的是明确展示了一个 GUI Agent 系统完整架构的论文——它由哪些模块组成、模块之间如何协作、数据如何流转、控制循环如何运转。比如 AppAgent、UFO、Agent S、OS-Copilot 等。

在 145 篇展示了完整系统架构的 GUI Agent 框架论文中，各模块的出现比例如下：

模块	论文数	占比	含义
感知	145	100%	观察界面
决策	145	100%	选择下一步做什么

执行	145	100%	把决策变成界面操作
验证	93	64.1%	检查动作是否真的产生了预期效果
记忆	88	60.7%	跨步骤保留历史信息
规划	79	54.5%	把全局任务分解为子任务
反思	61	42.1%	分析失败原因并调整策略

6.2 两种架构范式

模块化流水线

占比：47.1%（154/327 篇 SE 可解析论文）

把 GUI Agent 拆分成多个独立、可替换、可检查的组件，每个组件负责一个明确职责，组件之间通过接口协作。

典型结构：

```
观察适配器 → 感知/定位 → 规划/决策 → 执行器 → 环境
      ↑                                        ↓
      └──── 验证/记忆/反思 ←──────────────────┘
```

代表系统：Agent S、UFO、AppAgent、OS-Copilot

优势：可检查、可替换、可测试、可治理、可恢复

劣势：模块间协调成本高，接口设计复杂，端到端延迟可能增加

端到端原生 Agent

占比：仅 5.5%（18/327 篇 SE 可解析论文）

把感知、推理、决策、动作预测压缩进一个统一的多模态模型，不再拆分成独立模块。模型直接输入截图和指令，输出下一步动作。

典型结构：

```
截图 + 指令 → [统一多模态模型] → 下一步动作
```

代表系统：UI-TARS、OS-ATLAS、ShowUI、InfiGUIAgent

优势：接口紧凑、协调开销低、训练信号统一、部署简单

劣势：可解释性差、可替换性差、可测试性差

7 产业实践

国际产品：

- Anthropic Claude Computer Use：屏幕感知 + 光标键盘控制
- OpenAI Operator → ChatGPT Agent：浏览器 Agent，后转向分阶段集成
- Google Gemini 2.0 "agentic era"：浏览器使用作为行动系统组件
- Amazon Nova Act：浏览器工作流自动化，UI 交互 + 代码 + 工具

中国产业：

- 阿里 MobileAgent-v3 / GUI-Owl：大规模数据生成 + 在线 RL + 端云协同
- 字节 UI-TARS / UI-TARS-2：原生截图中心 Agent，多轮 RL
- 智谱 AutoGLM：自主基础 Agent
- 荣耀 YOYO、小米超级小爱：设备端跨应用操作

8 问题挑战

8.1 技术挑战

GUI Agent 目前有五个核心技术难题：

挑战	问题描述	可能的解决方向
状态感知执行	界面看起来成功了，实际后台没成功	跟踪真实状态、执行后检查结果、运行时监控
自适应感知	观察成本与信息完整性难以兼顾，缺乏何时看、看多少的决策依据	按需放大、按需请求结构信息、设定证据门槛
混合自动化	纯 GUI 操作脆弱，且可能产生不可逆副作用	GUI 与工具/API 混合执行、沙箱、回滚、危险操作确认
数据不匹配	训练与部署时的观察条件不一致（训练可见完整结构，部署仅能看截图）	补充验证、恢复、升级等环节的监督信号
基准基础设施	评测工具本身也是复杂软件，其可靠性直接影响结论	公开观察条件、增加任务是否真正完成的检查

8.2 软件工程缺口

一共统计了 327 篇论文，考察它们在软件工程实践上的覆盖情况，结果如下：

工程实践	覆盖情况	说明
基准/消融测试	约一半	多数研究仍以基准测试为主要验证方式
无效动作处理	不到三分之一	对点错、误操作等异常的处理仍不普遍
反思/重试/重规划	约四分之一	具备自我纠错能力的系统仍是少数
轨迹/日志/重放	很少	少有系统保存操作记录用于调试与审计
运行时人类监督	很少	人机协同多数停留在概念层面
维护流程	绝大多数缺失	几乎没有论文讨论系统上线后如何维护
监控/审计	绝大多数缺失	几乎没有系统具备运行时监控与审计能力

参考文献

1. Wu Y, Yang Y. A Platform-Aware Survey of Execution, Verification, and Risk of Graphical User Interface Agents. *Engineering Information Technology & Electronic Engineering*, 2026, 27(7): 260103.
2. Bai B, et al. Efficient GUI Agents: A Systems Survey of Observation, Memory, Action, and Runtime Optimization. arXiv preprint, 2026.
3. Hu J, Liu J, Lai J, et al. GUI Agents with Reinforcement Learning: Toward Digital Inhabitants. arXiv preprint, 2026.
4. 塔杰, 王亮, 刘进, 等. GUI-Agent：研究、应用与展望. *软件导刊*, 2025, 24(8): 1-10.
5. Nguyen D, Chen J, Wang Y, et al. GUI Agents: A Survey. Findings of the Association for Computational Linguistics: ACL 2025, 2025: 22522-22538.
6. Yu S, Ling Y, Xing J, et al. Software Engineering for and with GUI Agent. arXiv preprint, 2026.